

Schnellstart

Dieses Kapitel führt Sie in 5 Minuten zur ersten lauffähigen GEEF-Pipeline.

Voraussetzungen

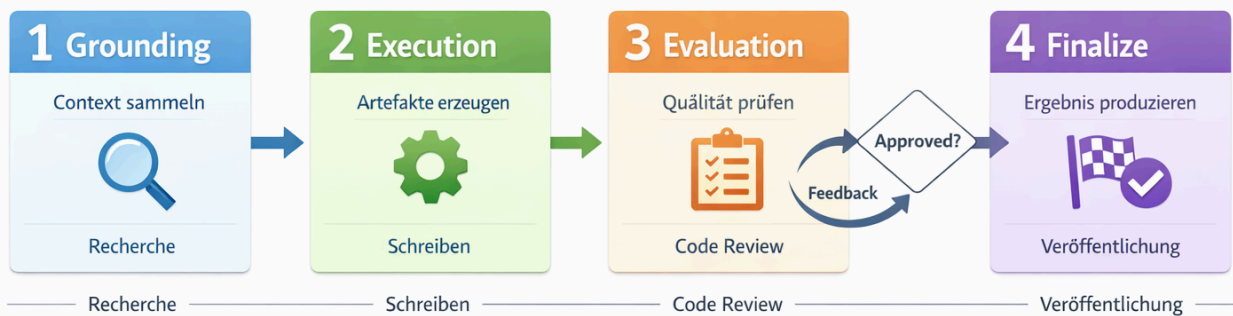
- .NET 8 SDK oder höher
- Ein beliebiger Code-Editor (VS Code, Rider, Visual Studio)

Installation

```
dotnet add package Geef.Sdk --version 1.0.0
```

Ihre erste Pipeline

Eine GEEF-Pipeline besteht aus vier Schritten:



Minimales Beispiel

```
using Geef.Sdk;

using Geef.Sdk.Context;
using Geef.Sdk.Providers;
using Geef.Sdk.Results;

// 1. Keys definieren
var artifactKey = new ContextKey<string>("my:artifact");

// 2. Pipeline zusammenbauen
var pipeline = Geef.CreatePipeline<string>()
    .UseGrounding(new MyGrounding(artifactKey))
    .UseExecution(new MyExecution(artifactKey))
    .AddReviewer(new MyReviewer(artifactKey))
    .UseFinalizer(new MyFinalizer(artifactKey))
    .Build();

// 3. Ausführen
var result = await pipeline.RunAsync("Schreibe eine Zusammenfassung über Quantencomputing");

Console.WriteLine($"Erfolg: {result.Success}");
Console.WriteLine($"Iterationen: {result.TotalIterations}");
Console.WriteLine($"Ergebnis: {result.Output}");
```

Grounding — Kontext sammeln

```
public class MyGrounding(ContextKey<string> artifactKey) : IGroundingStep
{
    public Task<GroundingResult> RunAsync(string input, CancellationToken ct = default)
    {
        // Hier: RAG-Abfragen, Dateien lesen, APIs aufrufen, etc.
        var context = new RunContext()
            .Set(artifactKey, string.Empty); // Platzhalter für das Artefakt

        return Task.FromResult(new GroundingResult { Context = context });
    }
}
```

Execution — Artefakte erzeugen

```
public class MyExecution(ContextKey<string> artifactKey) : IExecutionStep
{
    public async Task<ExecutionResult> RunAsync(IRunContext context, CancellationTok
en ct = default)
    {
        // Vorherige Findings lesen (ab Iteration 2)
        var previousFindings = context.TryGet(GeefKeys.PreviousFindings, out var find
ings)

        ? findings
        : Array.Empty<Finding>();

        // Artefakt generieren (hier: LLM-Aufruf)
        var prompt = previousFindings.Count > 0
            ? $"Überarbeite basierend auf: {string.Join(", ", previousFindings.Select
(f => f.Message))}"
            : "Erstelle eine neue Zusammenfassung";

        var generatedText = await CallYourLlmAsync(prompt, ct);

        // Neuen, unveränderlichen Context zurückgeben
        return new ExecutionResult
        {
            UpdatedContext = context.Set(artifactKey, generatedText)
        };
    }
}
```

Reviewer — Qualität prüfen

```
public class MyReviewer(ContextKey<string> artifactKey) : IReviewer
{
    public string Name => "QualityReviewer";

    public async Task<ReviewResult> ReviewAsync(IRunContext context, CancellationToken ct = default)
    {
        var artifact = context.GetRequired(artifactKey);

        // Prüfung: Mindestlänge
        if (artifact.Length < 100)
        {
            return new ReviewResult
            {
                ReviewerName = Name,
                Decision = ReviewDecision.Rejected,
                Findings = new[]
                {
                    new Finding
                    {
                        ReviewerName = Name,
                        Fingerprint = "too-short",
                        Message = "Text ist zu kurz (< 100 Zeichen)",
                        Severity = FindingSeverity.Error
                    }
                },
                Duration = TimeSpan.Zero
            };
        }

        return new ReviewResult
        {
            ReviewerName = Name,
            Decision = ReviewDecision.Approved,
            Duration = TimeSpan.Zero
        };
    }
}
```

Finalizer — Ergebnis produzieren

```
public class MyFinalizer(ContextKey<string> artifactKey) : IFinalizer<string>
{
    public Task<FinalizeResult<string>> FinalizeAsync(IRunContext context, Cancellati
onToken ct = default)
    {
        return Task.FromResult(new FinalizeResult<string>
        {
            Output = context.GetRequired(artifactKey),
            FinalContext = context,
            Summary = "Zusammenfassung erfolgreich generiert"
        });
    }
}
```

Was passiert intern?

Iteration 1:

Grounding → Kontext initialisiert

Execution → LLM generiert Text (z.B. 50 Zeichen)

Evaluation → Reviewer: "Rejected – zu kurz"

→ PreviousFindings werden gesetzt, nächste Iteration

Iteration 2:

Execution → LLM überarbeitet (liest PreviousFindings)

Evaluation → Reviewer: "Approved"

→ Finalize → Ergebnis: der generierte Text

Nächste Schritte

- Konzepte und Architektur — Den GEEF-Loop im Detail verstehen
- Konfiguration und Policies — Konvergenz, Strategien und Middleware
- API-Referenz — Alle Typen und Methoden